

Curso: Sistemas de Informação	
Disciplina: Mapeamento Objeto Relacional	Período: 4
Professor: Luiz Gustavo Dias	Tipo: Material Complementar
Objetivo: Consulta e Delete	

Pré-requisito: necessário ter uma aplicação Java implementada com o uso de JPA com Maven e Hibernate e objetos persistidos em banco de dados.

REALIZANDO CONSULTAS SIMPLES

A realização de consultas simples em um projeto Java com Maven e Hibernate, que faça uso do EntityManager, se dá através do método 'find'.

Esse método deve ser executado a partir de um objeto EntityManager, assim como procedido com o método 'persist' em uma etapa anterior.

Contudo, antes de implementá-lo é necessário incluir um construtor sem argumentos na classe que representa o objeto.

No JPA/Hibernate, o construtor sem argumentos é necessário porque o framework precisa criar automaticamente uma instância da entidade quando recupera um registro do banco de dados; nesse momento, ele primeiro cria um objeto Produto vazio e, em seguida, preenche seus atributos com os valores encontrados no banco.

Diferentemente do construtor parametrizado, que é utilizado pelo programador para criar um novo produto informando seus dados, o construtor vazio permite que o Hibernate faça esse processo de reconstrução do objeto de forma automática, sendo, portanto, um requisito da entidade JPA para que ela possa ser corretamente gerenciada pelo mecanismo de persistência.

```
/*Construtor vazio necessário ao Hibernate  
*  
*/  
public Produto() {  
  
}
```

Retomemos agora o método find.

Na chamada do método é necessário passar como parâmetro dois componentes:

- a) **Tabela.class**: corresponde à tabela a ser consultada;
- b) **Variável que se deseja localizar**: corresponde ao valor atribuído à pk da tupla.

Exemplo:

```
em.getTransaction().begin();  
em.find(Produto.class, 1);  
em.close();  
emf.close();|
```

Para que o resultado da busca seja impresso você pode passar o método find como parâmetro do sysout. Dessa forma, o registro da tupla será impresso integralmente:

```
//imprime o produto integralmente  
System.out.println(em.find(Produto.class, 1));
```

Para que o resultado da consulta esteja disponível em tempo de execução do software é necessário instanciar um objeto correspondente ao tipo de registro consultado e utilizá-lo para armazenar o resultado:

```
//armazena toda a tupla em uma variável do tipo produto  
Produto x = em.find(Produto.class, 1);
```

Após armazenado, pode-se acessar atributos específicos para realizar outros procedimentos, como a impressão na tela apenas do nome do produto (considerando o exemplo deste conteúdo):

```
//imprime um atributo específico  
System.out.println("Nome: " + x.getNome());
```

Utilizando o Scanner

Em um contexto mais amplo, é possível utilizar a classe Scanner para que seja promovida interação com o usuário no momento da consulta de um objeto.

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("exemplo-jpa");
EntityManager em = emf.createEntityManager();

Scanner scanner = new Scanner(System.in);

System.out.println("Informe o id do produto: ");
Integer id = scanner.nextInt();

em.getTransaction().begin();

//imprime o produto integralmente
System.out.println(em.find(Produto.class, id));

//armazena toda a tupla em uma variável do tipo produto
Produto x = em.find(Produto.class, id);

//imprime um atributo específico
System.out.println("Nome: " + x.getNome());

em.close();
emf.close();
```

REMOVENDO REGISTROS

A remoção de registros ocorre através do método 'remove' do EntityManager. Sua aplicação é similar ao método 'persist', devendo ser passado um objeto existente para que seja removido da tabela.

Consideremos o exemplo anterior, no qual armazenamos uma tupla consultada na variável x do tipo Produto.

Suponhamos que desejamos excluir este registro:

```
//remover um registro (descomentar)
em.remove(x);

em.getTransaction().commit();

em.close();
emf.close();
```

Oberve que, além de utilizar o 'remove' é necessário utilizar o 'getTransaction().commit()' para proceder com a remoção do registro, pois estamos realizando uma modificação no banco de dados.

Por que não foi preciso usar o 'getTransaction().commit()' ao proceder com o find?

Uma simples consulta ao banco não promove modificações, portanto, não há alteração a ser confirmada.